

PROJECT REPORT

Restaurant Database Management System

A Relational Database Design & Implementation using SQL

Database: SingleRestaurantDB

Platform: MySQL

By Muhammad Ameer Abdullah

1. Introduction

The Restaurant Database Management System is a relational database designed to digitize and streamline the core operations of a single restaurant. Rather than relying on paper records or scattered spreadsheets, the system provides one centralized MySQL database that captures customer information, the food menu, orders placed, the individual items within each order, and the payments made against those orders.

The database, named SingleRestaurantDB, was built using standard SQL Data Definition Language (DDL) and Data Manipulation Language (DML) statements. It follows a normalized relational structure so that data such as customer details and menu items are stored once and referenced wherever needed, reducing redundancy and keeping the data consistent.

2. Objectives

- Design a normalized relational schema that accurately reflects restaurant operations.
- Maintain accurate records of customers, menu items, orders, order line items, and payments.
- Enforce data integrity between related tables using primary and foreign keys.
- Enable quick retrieval of business information, such as order history and total sales, through SQL queries.
- Provide a foundation that can later be extended into a full application with a front-end interface.

3. Tools & Technologies Used

- Database Engine: MySQL
- Language: Structured Query Language (SQL)
- Design Approach: Relational Database Design with Primary/Foreign Key constraints

4. Database Design

The database consists of five interrelated tables. Customers and Food_Menu act as independent master tables, while Orders, Order_Details, and Payments are transactional tables that reference them. This separation keeps each order lightweight while still allowing the system to trace every item sold and every payment collected back to a specific customer and order.

4.1 Entity Relationship Overview

- A Customer can place many Orders (one-to-many).

- An Order can contain many Food items via Order_Details, and each Food item can appear in many Order_Details rows (many-to-many, resolved through the Order_Details junction table).
- An Order has exactly one Payment record (one-to-one in this dataset).

4.2 Table Schemas

Table: Customers

Column	Data Type	Key	Description
customer_id	INT (AUTO_INCREMENT)	Primary Key	Unique identifier for each customer
name	VARCHAR(100)	-	Full name of the customer
phone	VARCHAR(15)	-	Contact phone number
email	VARCHAR(100)	-	Email address
address	VARCHAR(255)	-	Delivery/billing address

Table: Food_Menu

Column	Data Type	Key	Description
food_id	INT (AUTO_INCREMENT)	Primary Key	Unique identifier for each menu item
food_name	VARCHAR(100)	-	Name of the dish
category	VARCHAR(50)	-	Menu category (e.g., Burger, Pizza)
price	DECIMAL(10,2)	-	Unit price of the item
availability	VARCHAR(20)	-	Availability status of the item

Table: Orders

Column	Data Type	Key	Description
order_id	INT (AUTO_INCREMENT)	Primary Key	Unique identifier for each order
customer_id	INT	Foreign Key -> Customers	Customer who placed the order
order_date	DATE	-	Date the order was placed
total_amount	DECIMAL(10,2)	-	Total value of the order
status	VARCHAR(20)	-	Order status (e.g., Delivered)

Table: Order_Details

Column	Data Type	Key	Description
detail_id	INT (AUTO_INCREMENT)	Primary Key	Unique identifier for each line item
order_id	INT	Foreign Key -> Orders	Order this line item belongs to
food_id	INT	Foreign Key -> Food_Menu	Food item ordered
quantity	INT	-	Quantity of the item ordered
subtotal	DECIMAL(10,2)	-	Price for this line (quantity x price)

Table: Payments

Column	Data Type	Key	Description
payment_id	INT (AUTO_INCREMENT)	Primary Key	Unique identifier for each payment
order_id	INT	Foreign Key -> Orders	Order the payment is associated with
payment_method	VARCHAR(20)	-	Method used (e.g., Cash)
payment_status	VARCHAR(20)	-	Payment status (e.g., Paid)

5. Sample Data

The following records were inserted to populate and test the database.

5.1 Customers

customer_id	name	phone	email	address
1	Ali	03001234567	ali@gmail.com	Abbottabad
2	Sara	03123456789	sara@gmail.com	Islamabad

5.2 Food_Menu

food_id	food_name	category	price	availability
1	Zinger Burger	Burger	500.00	Available
2	Chicken Pizza	Pizza	1200.00	Available
3	Fries	Sides	200.00	Available

5.3 Orders

order_id	customer_id	order_date	total_amount	status
1	1	2026-05-20	1700.00	Delivered

5.4 Order_Details

detail_id	order_id	food_id	quantity	subtotal
1	1	1	1	500.00
2	1	2	1	1200.00

5.5 Payments

payment_id	order_id	payment_method	payment_status
1	1	Cash	Paid

6. Queries and Results

Beyond basic table creation and data entry, several SQL queries were written to demonstrate how the database answers real business questions using JOIN operations and aggregate functions.

6.1 Customer Order Summary

This query joins Customers with Orders to show which customer placed which order, along with the order total and status.

```
SELECT c.name, o.order_id, o.total_amount, o.status FROM Customers c JOIN Orders o ON c.customer_id = o.customer_id;
```

name	order_id	total_amount	status
Ali	1	1700.00	Delivered

6.2 Order Line-Item Detail

This query joins Orders, Order_Details, and Food_Menu to break each order down into the individual food items, quantities, and subtotals.

```
SELECT o.order_id, f.food_name, od.quantity, od.subtotal FROM Orders o JOIN Order_Details od ON o.order_id = od.order_id JOIN Food_Menu f ON od.food_id = f.food_id;
```

order_id	food_name	quantity	subtotal
1	Zinger Burger	1	500.00
1	Chicken Pizza	1	1200.00

6.3 Total Sales

This aggregate query calculates the total revenue generated across all orders in the system.

```
SELECT SUM(total_amount) AS total_sales FROM Orders;
```

total_sales
1700.00

7. Conclusion

This project successfully demonstrates the design and implementation of a normalized relational database for managing single-restaurant operations. The five-table structure cleanly separates customer data, menu data, and transactional data, while primary and foreign key constraints preserve referential integrity across the system. The sample queries confirm that the schema supports common business needs such as viewing order history, itemized order breakdowns, and total sales reporting.

8. Future Enhancements

- Add a Staff/Employee table to track who processed each order.
- Introduce a Reservations table for table bookings.
- Build a front-end application (web or desktop) on top of this schema.
- Add stored procedures and triggers, e.g., to auto-update total_amount when order details change.
- Extend the schema to support multiple restaurant branches.